

Atividade Prática: Algoritmos de Mudança de Base

Cálculo Numérico – EXA1132
Bacharelado em Ciência da Computação
Universidade Federal Rural do Semi-Árido – UFERSA

2026.2

1 Contextualização

Na aula sobre *Sistemas de Numeração e Erros em Processos Numéricos*, vimos o **Teorema da Mudança de Base**, que garante que todo número natural N admite representação única em qualquer base $b \geq 2$, na forma

$$N = \sum_{i=0}^n a_i b^i, \quad 0 \leq a_i \leq b - 1.$$

Vimos também que esse teorema é *construtivo*: sua demonstração já fornece dois algoritmos práticos — um para converter um número de uma base qualquer para decimal (por expansão polinomial), e outro para converter um número decimal para uma base qualquer (por divisões sucessivas). Esta atividade pede que vocês implementem esses dois algoritmos.

2 Objetivos de Aprendizagem

Ao final desta atividade, o aluno deve ser capaz de:

- Traduzir a demonstração matemática do Teorema da Mudança de Base em um algoritmo computável;
- Implementar corretamente aritmética posicional para bases $\beta \in \{2, 3, \dots, 36\}$;
- Tratar casos de borda (base inválida, dígitos inválidos, número zero, números negativos);
- Validar a correteza do próprio código por meio de casos de teste.

3 Enunciado

Implemente **dois algoritmos** (funções ou programas independentes), na linguagem de sua preferência (Python, C, C++ ou Java), descritos a seguir.

Algoritmo 1 – De uma base β qualquer para a base 10

Entrada: uma base de origem β , com $2 \leq \beta \leq 36$, e uma string representando um número nessa base, usando os dígitos 0–9 e, quando necessário, as letras A–Z (maiúsculas) para valores de 10 a 35.

Saída: o valor equivalente do número em base decimal (base 10).

Método esperado: aplicar diretamente a expansão polinomial

$$N_{10} = \sum_{i=0}^n a_i \beta^i,$$

percorrendo os dígitos da string (do mais significativo ao menos significativo) e acumulando o resultado – não é permitido usar funções prontas da linguagem que já façam a conversão de base (ex.: `int(s, base)` em Python).

Algoritmo 2 – Da base 10 para uma base β qualquer

Entrada: um número inteiro não negativo N em decimal e uma base de destino β , com $2 \leq \beta \leq 36$.

Saída: uma string representando N na base β , usando os dígitos 0–9 e A–Z quando aplicável.

Método esperado: aplicar o algoritmo de divisões inteiras sucessivas por β , guardando os restos e montando a string final na ordem correta (do resto mais recente/mais significativo para o mais antigo/menos significativo). Tratar corretamente o caso $N = 0$.

4 Requisitos Técnicos

1. As duas funções devem validar a base informada e emitir uma mensagem de erro clara se $\beta < 2$ ou $\beta > 36$;
2. O Algoritmo 1 deve validar que todos os caracteres da string de entrada são dígitos válidos para a base β informada (ex.: o dígito '8' é inválido em base 8);
3. Deve ser incluída uma pequena rotina de **teste automático** (função `main` ou script separado) que execute os casos de teste da Seção 5 e imprima se cada um passou ou falhou;
4. O código deve ser comentado, indicando claramente onde ocorre cada etapa do algoritmo (expansão polinomial ou divisões sucessivas);
5. Não é permitido o uso de bibliotecas ou funções que já resolvam a conversão de base diretamente (ex.: `bin()`, `hex()`, `oct()`, `int(s, base)` em Python; `Integer.toString(n, base)` em Java).

5 Casos de Teste Sugeridos

Algoritmo	Entrada	Saída esperada	Observação
1	$(1101)_2$	13	binário simples
1	$(2F)_{16}$	47	hexadecimal com letra
1	$(377)_8$	255	octal
1	$(Z)_{36}$	35	base máxima, um dígito
1	$(0)_5$	0	caso de borda
2	45, $\beta = 2$	$(101101)_2$	já visto em aula
2	255, $\beta = 16$	$(FF)_{16}$	hexadecimal
2	0, $\beta = 7$	$(0)_7$	caso de borda
2	1000, $\beta = 36$	$(RS)_{36}$	base máxima

Sugestão: depois de implementar os dois algoritmos, teste-os em **composição**: converta um número de base 10 para base β com o Algoritmo 2 e, em seguida, converta o resultado de volta para base 10 com o Algoritmo 1. Se tudo estiver correto, você deve recuperar o número original – essa é uma boa forma de autovalidar seu código.

6 Forma de Entrega

- Um arquivo de código-fonte por algoritmo (ou um único arquivo com as duas funções bem separadas e identificadas);
- Um pequeno relatório (máximo 1 página) descrevendo: linguagem escolhida, decisões de implementação, e resultado dos testes da Seção 5;
- Entrega via SIGAA, na atividade correspondente, até a data informada em aula.

7 Critérios de Avaliação

Critério	Peso
Algoritmo 1 correto (base $\beta \rightarrow 10$)	30%
Algoritmo 2 correto ($10 \rightarrow$ base β)	30%
Tratamento de casos de borda e validação de entrada	15%
Casos de teste implementados e corretos	15%
Clareza do código e do relatório	10%

8 Referência

Conteúdo baseado na aula *Sistemas de Numeração e Erros em Processos Numéricos* (Teorema da Mudança de Base e seu Corolário), ministrada na disciplina EXA1132 – Cálculo Numérico.